

Práctico 5

Gestión de la memoria y memoria virtual

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento presenta la resolución integral del Práctico 5 de la materia Sistemas Operativos (UNRC, 2026), centrado en la *gestión de la memoria* y la *memoria virtual*. Se resuelven nueve ejercicios que cubren: inspección del consumo de memoria con `top`, `free`, `smem` y `swapon`; *buddy system*; análisis del layout lógico de un proceso C; arquitecturas con paginado (tabla de páginas, formato de *pte*, páginas grandes vs. multinivel); memoria compartida entre procesos; mapeo de archivos en memoria (`mmap`); y la API de *shared memory* de UNIX (`shmget`, `shmat`, `shmdt`, `shmctl`). Cada ejercicio se justifica con los capítulos *Gestión de la memoria* y *Memoria virtual* de las Notas del curso (Marcelo Arroyo) y con los resúmenes teóricos `resumen-teorico-gestion-memoria` y `resumen-teorico-memoria-virtual` disponibles en `materiales/`.

Índice

1. Introducción	2
2. Ejercicio 1 — Memoria y swap usados por los procesos	2
2.1. Resumen global con <code>free</code>	3
2.2. Dispositivos de swap con <code>swapon</code>	3
2.3. Vista por proceso con <code>top</code>	3
2.4. Vista detallada con <code>smem</code>	4
2.5. Información cruda por proceso	4
3. Ejercicio 2 — Evolución de un heap con <i>buddy system</i>	5
3.1. Recordatorio del algoritmo	5
3.2. Operación <code>ptr1 = malloc(20)</code>	5
3.3. Operación <code>ptr2 = malloc(50)</code>	5
3.4. Operación <code>ptr3 = malloc(1010)</code>	6
3.5. Operación <code>free(ptr1)</code>	6
3.6. Operación <code>malloc(64)</code>	6
4. Ejercicio 3 — Direcciones lógicas de cada área	6
4.1. Programa propuesto	7
4.2. Salida típica	7
4.3. Lectura del layout	7
4.4. Verificación con <code>/proc/<pid>/maps</code>	8
5. Ejercicio 4 — Arquitectura de 16 bits con páginas de 1 KB	8
5.1. 4.a — Capacidad máxima de la memoria	8
5.2. 4.b — Cantidad de páginas	9
5.3. 4.c — Representación de la tabla de páginas y de la <i>pte</i>	9
5.4. 4.d — Tabla de páginas de un proceso concreto	9

6. Ejercicio 5 — Tablas de páginas para 4 GB en 32 bits	10
6.1. 5.a — Páginas de 4 KB	10
6.2. 5.b — Páginas de 4 MB	11
7. Ejercicio 6 — Representación de la tabla de páginas de un proceso	11
8. Ejercicio 7 — Memoria compartida entre dos procesos	12
8.1. Análisis de las direcciones lógicas	12
8.2. Diagrama	12
8.3. Tablas de páginas	12
9. Ejercicio 8 — mmap de un archivo	13
9.1. Listado del programa original	13
9.2. Compilación y ejecución	14
9.3. 8.a — ¿Por qué <code>data + 7000</code> no causa <code>segfault</code> ?	14
9.4. 8.b — Modificación para lectura y escritura con persistencia	14
9.5. 8.c — Tamaño del archivo después	15
10. Ejercicio 9 — Memoria compartida IPC con <code>shmwriter</code> y <code>shmreader</code>	16
10.1. Cómo funciona la API	16
10.2. Programas provistos	16
10.3. Ejecución base	17
10.4. 9.a — Volver a correr <code>shmreader</code>	17
10.5. 9.b — Comentar la destrucción y correr varias veces	18
10.6. 9.c — Lectura/escritura en <code>str + 500</code>	18
11. Conclusiones	19

1. Introducción

La gestión de la memoria es una de las funciones más importantes que realiza el sistema operativo. Sus dos responsabilidades inseparables son la **asignación** (decidir qué regiones de la RAM le tocan a qué procesos y al kernel, y cómo se liberan cuando ya no se usan) y la **protección** (garantizar que un proceso se ejecute confinado a sus regiones, sin poder invadir las de otros procesos ni las del kernel).

A su vez, la **memoria virtual** es la técnica que permite ejecutar programas cuya memoria total excede la RAM disponible, manteniendo en RAM sólo las partes en uso en cada momento y dejando el resto en disco. Se apoya en el esquema de *paginado* y en bits de control de la **pte** (V, A, D) para implementar *swapping/demand paging*, **mmap**, crecimiento dinámico del stack, memoria compartida entre procesos y *copy on write*.

Este documento aplica esos conceptos a problemas concretos: desde inspeccionar el consumo real del sistema hasta diseñar tablas de páginas para una arquitectura dada, pasando por la simulación de un *buddy allocator* y el análisis empírico de **mmap** y la memoria compartida en Linux.

2. Ejercicio 1 — Memoria y swap usados por los procesos

Enunciado. Determinar los espacios de memoria (incluyendo swap) que están usando los procesos del sistema. Ayuda: ver los comandos `top`, `free`, `smem` y `swapon`.

2.1. Resumen global con free

`free` presenta una visión *global* del sistema en una sola línea: cuánta RAM y cuánto *swap* hay, cuánto está usado, libre, compartido y cuánto se ocupa en *buffers/cache*.

```
$ free -h
              total usado libre compart. buff/cache disponible
Mem: 7.6Gi 3.1Gi 980Mi 340Mi 3.6Gi 3.8Gi
Swap: 2.0Gi 512Mi 1.5Gi
```

Interpretación de columnas:

- **total**: capacidad total de RAM (o swap) reconocida por el kernel.
- **usado**: RAM efectivamente asignada a procesos, kernel y reservada.
- **libre**: páginas libres (con bit *V* apagado en ningún *pte* que las refiera).
- **compart.:** páginas de *shared memory* (`tmpfs`, `IPC`, `mmap MAP_SHARED`).
- **buff/cache**: páginas usadas por el cache de disco (*page cache*). Son liberables: el kernel las descarta si la RAM escasea.
- **disponible**: estimación de cuánta memoria se puede asignar sin recurrir a swap (esencialmente `libre + buff/cache reclamable`).

2.2. Dispositivos de swap con swapon

`swapon -show` lista los dispositivos o archivos configurados como swap. Cada entrada indica tipo (*partition* o *file*), tamaño y cuánto se está usando:

```
$ swapon --show
NAME TYPE SIZE USED PRIO
/dev/sda3 partition 2.0G 512M -2
```

Una *partición* con formato de tipo *swap* es más rápida que un *archivo* swap, porque evita la indirección del sistema de archivos.

2.3. Vista por proceso con top

`top` muestra los procesos en tiempo real. Las columnas relevantes para memoria son:

Columna	Significado
VIRT	Tamaño total del espacio de direcciones lógicas del proceso (texto, datos, heap, stack, <code>mmap</code> , librerías compartidas, etc.).
RES	<i>Resident Set Size</i> : páginas que efectivamente están en RAM.
SHR	Páginas residentes que son <i>compartidas</i> con otros procesos (típicamente librerías <code>.so</code> , <code>mmap MAP_SHARED</code>).
%MEM	RES dividido por la RAM total.
SWAP	Páginas del proceso que actualmente residen en el área de swap (sólo si se habilita la columna con <code>f</code> en <code>top</code>).

Importante

VIRT *no* representa memoria realmente consumida: incluye *mappings* (por ejemplo, librerías compartidas) que pueden no estar en RAM. Sumar VIRT de todos los procesos puede dar varias veces la RAM física y no significa que el sistema esté *out of memory*.

2.4. Vista detallada con smem

smem es la herramienta más precisa para conocer el consumo *real* de cada proceso, porque introduce las métricas **USS** y **PSS**:

- **USS** (*Unique Set Size*): memoria que sólo usa el proceso. Es lo que se liberaría si el proceso terminara.
- **PSS** (*Proportional Set Size*): RAM única más las páginas compartidas *prorrateadas* entre los procesos que las comparten. La suma de PSS de todos los procesos da el consumo real de memoria de usuario.
- **RSS** (*Resident Set Size*): RAM total mapeada por el proceso (cuenta cada página compartida en cada proceso que la usa).
- **SWAP**: páginas del proceso en el área de swap.

```
$ smem -tk -c "pid user command swap uss pss rss"
PID User Command Swap USS PSS RSS
1234 tomi /usr/bin/firefox 12M 320M 430M 920M
4567 tomi /usr/lib/code/code-helper 8M 90M 140M 400M
...
-----
128M 1.1G 1.6G 3.4G
```

Idea intuitiva

RSS sobreestima el consumo porque cuenta varias veces la misma página compartida; USS subestima porque ignora las compartidas; PSS es el mejor compromiso. Cuando se pregunta “¿cuánta RAM consume este proceso de verdad?”, la respuesta más honesta es PSS.

2.5. Información cruda por proceso

El kernel expone los datos por proceso en `/proc/<pid>/status` y `/proc/<pid>/smaps`:

```
$ grep -E "VmSize|VmRSS|VmSwap|VmData|VmStk|VmExe|VmLib" /proc/$$/status
VmSize: 23012 kB # tamaño virtual total
VmRSS: 4140 kB # resident set size
VmData: 1872 kB # segmento de datos
VmStk: 132 kB # stack
VmExe: 872 kB # código (text)
VmLib: 3568 kB # librerías compartidas
VmSwap: 256 kB # páginas en swap
```

El archivo `/proc/<pid>/smaps` lista cada VMA (región) por separado con campos como **Size**, **Rss**, **Pss**, **Swap**, **Shared_Clean** y **Shared_Dirty**, entre otros.

Conclusión

- **free** y **swapon** dan el panorama global del sistema.
- **top** ordena los procesos por consumo aparente (**VIRT/RES**) pero no diferencia memoria compartida.
- **smem** agrega **PSS/USS** y es lo más adecuado para responder por proceso.
- `/proc/<pid>/status` y `smaps` son la fuente cruda que usan todas las herramientas anteriores.

3. Ejercicio 2 — Evolución de un heap con *buddy system*

Enunciado. Mostrar la evolución de un heap de tamaño 1024 (2^{10}) gestionado con un *buddy system* con bloques de tamaño entre 2^5 y 2^{10} ante la secuencia `ptr1=malloc(20)`, `ptr2=malloc(50)`, `ptr3=malloc(1010)`, `free(ptr1)` y `malloc(64)`.

Sobre el tamaño del heap. El enunciado original menciona “1 MB”, pero el rango de bloques permitidos es 2^5 (32 bytes) a 2^{10} (1024 bytes). La interpretación más coherente con esa restricción es que el heap tiene tamaño $2^{10} = 1024$ bytes (es decir, un único bloque máximo). Así, todas las operaciones se pueden expresar en términos de los tamaños permitidos y la subdivisión del bloque maestro de 1024 bytes.

3.1. Recordatorio del algoritmo

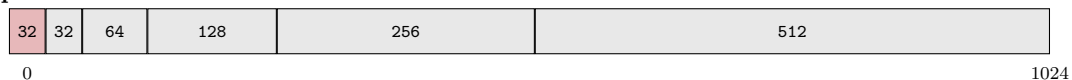
- Inicialmente, el heap se particiona en bloques del tamaño máximo 2^h (1024 en este caso).
- Ante un pedido de tamaño n : se busca el menor bloque libre b con $b.size \geq 2^{\lceil \log_2 n \rceil}$. Si $b.size \geq 2n$ y $b.size > 2^l$, se divide en dos mitades (*buddies*) y se repite el paso. Finalmente se asigna b .
- Ante una liberación: si el *buddy* contiguo del mismo tamaño también está libre, se fusionan en un bloque del doble de tamaño. La fusión se repite mientras se pueda y sin pasar de 2^h .

3.2. Operación `ptr1 = malloc(20)`

20 cabe en el menor bloque permitido: $2^5 = 32$ bytes. El único bloque libre es el de 1024, así que hay que dividir sucesivamente hasta llegar a 32:

Paso	Acción	Estado del heap (tamaños)
0	inicio	[1024 _L]
1	divido 1024	[512 _L 512 _L]
2	divido el primer 512	[256 _L 256 _L 512 _L]
3	divido el primer 256	[128 _L 128 _L 256 _L 512 _L]
4	divido el primer 128	[64 _L 64 _L 128 _L 256 _L 512 _L]
5	divido el primer 64	[32 _L 32 _L 64 _L 128 _L 256 _L 512 _L]
6	asigno el primer 32 a ptr1	[32_U 32 _L 64 _L 128 _L 256 _L 512 _L]

ptr1



`ptr1` apunta al offset 0.

3.3. Operación `ptr2 = malloc(50)`

50 cabe en $2^6 = 64$ bytes. Hay un bloque libre de exactamente 64 (offset 64), así que se asigna directamente sin más divisiones.

ptr1 ptr2



`ptr2` apunta al offset 64.

3.4. Operación `ptr3 = malloc(1010)`

1010 necesita un bloque de $2^{10} = 1024$ bytes. Pero el bloque inicial de 1024 ya fue subdividido y sus pedazos no están disponibles en forma contigua (parte está usada por `ptr1` y `ptr2`). La suma de los bloques libres es $32 + 128 + 256 + 512 = 928$, y aún si fuera mayor, en *buddy system* los pedidos se satisfacen con un único bloque contiguo.

Importante

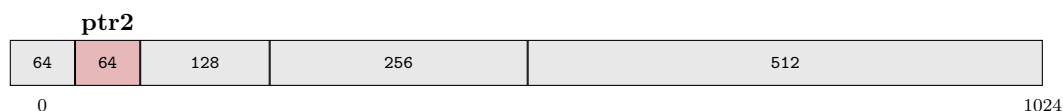
`ptr3 = malloc(1010)` falla y retorna NULL. Es un caso clásico de *fragmentación externa*: hay memoria libre, pero no agrupable en un bloque suficiente.

El estado del heap no cambia con este pedido fallido.

3.5. Operación `free(ptr1)`

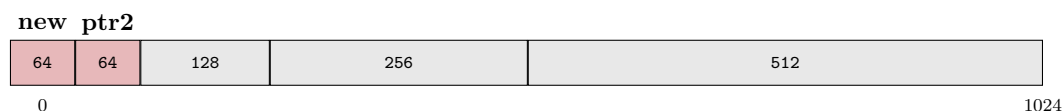
Se libera el bloque de 32 en el offset 0. Su *buddy* es el bloque de 32 en el offset 32, que también está libre: se **fusionan** en un único bloque de 64 en el offset 0.

¿El nuevo bloque de 64 tiene un *buddy* libre para seguir fusionando? Su *buddy* es el bloque de 64 en el offset 64 (donde está `ptr2`), que está **usado**. La fusión se detiene.



3.6. Operación `malloc(64)`

$64 = 2^6$. Hay un bloque libre de exactamente 64 en el offset 0, así que se asigna directamente.



Resumen

Operación	Resultado	Estado del heap (tamaños)
<code>ptr1=malloc(20)</code>	OK, asigna 32	$[32_U 32_L 64_L 128_L 256_L 512_L]$
<code>ptr2=malloc(50)</code>	OK, asigna 64	$[32_U 32_L 64_U 128_L 256_L 512_L]$
<code>ptr3=malloc(1010)</code>	FALLA (frag. ext.)	(sin cambios)
<code>free(ptr1)</code>	libera y fusiona 32+32	$[64_L 64_U 128_L 256_L 512_L]$
<code>malloc(64)</code>	OK, asigna 64	$[64_U 64_U 128_L 256_L 512_L]$

Idea intuitiva

El *buddy system* simplifica la administración (cada bloque tiene un único *buddy* fácil de calcular: basta con invertir un bit del offset) a costa de *fragmentación interna* (cada pedido se redondea al alza a la siguiente potencia de 2). El ejercicio muestra ambos costos: `malloc(20)` consume 32 bytes (12 desperdiciados, fragmentación *interna*) y `malloc(1010)` falla aunque haya 928 libres (fragmentación *externa*).

4. Ejercicio 3 — Direcciones lógicas de cada área

Enunciado. Hacer un programa en Linux (o MS-Windows) que permita analizar en qué direcciones lógicas se encuentran las áreas de código, datos, heap y stack.

4.1. Programa propuesto

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int dato_inicializado = 42; // .data
int dato_no_inicializado; // .bss
const char *literal = "hola"; // .rodata (apunta a literal en .text/.rodata)

void funcion(void) { }

int main(int argc, char **argv) {
    int local = 7; // stack
    int *heap_ptr = (int *)malloc(sizeof(int)); // heap

    printf("PID : %d\n", getpid());
    printf("---- code (.text) ----\n");
    printf("main() : %p\n", (void *)main);
    printf("funcion() : %p\n", (void *)funcion);
    printf("literal de cadena : %p \"%s\"\n", (void *)literal, literal);
    printf("---- data ----\n");
    printf("&dato_inicializado : %p (.data)\n", (void *)&dato_inicializado);
    printf("&dato_no_inicializado: %p (.bss)\n", (void *)&dato_no_inicializado);
    printf("---- heap ----\n");
    printf("malloc() : %p\n", (void *)heap_ptr);
    printf("brk actual : %p\n", sbrk(0));
    printf("---- stack ----\n");
    printf("&local : %p\n", (void *)&local);
    printf("&argc : %p\n", (void *)&argc);

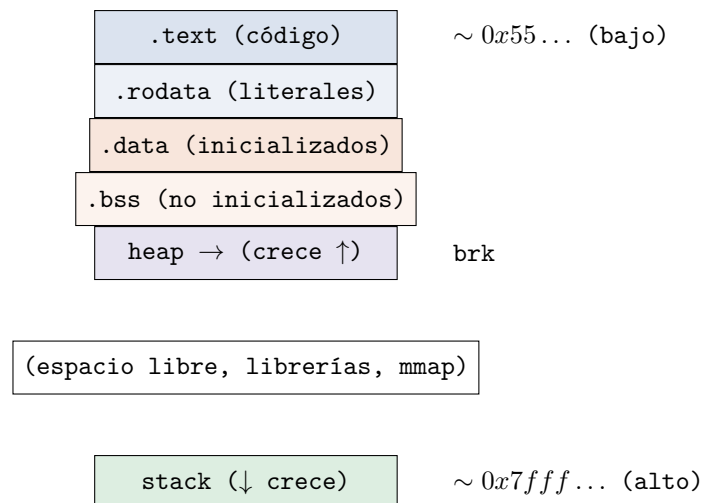
    free(heap_ptr);
    return 0;
}
```

4.2. Salida típica

```
PID : 18293
---- code (.text) ----
main() : 0x55a3f4d2b189
funcion() : 0x55a3f4d2b169
literal de cadena : 0x55a3f4d2c008 "hola"
---- data ----
&dato_inicializado : 0x55a3f4d2e010 (.data)
&dato_no_inicializado: 0x55a3f4d2e018 (.bss)
---- heap ----
malloc() : 0x55a3f5e6c2a0
brk actual : 0x55a3f5e8d000
---- stack ----
&local : 0x7fff89a3c2cc
&argc : 0x7fff89a3c2bc
```

4.3. Lectura del layout

Las direcciones se ordenan de menor a mayor de la siguiente forma:



- `main`, función y los literales (`.text`, `.rodata`) están en la *base baja* del proceso. Son de sólo lectura y ejecutables.
- `dato_inicializado` (`.data`) y `dato_no_inicializado` (`.bss`) están justo después, en la región de datos. `.bss` ocupa páginas en cero rellenas por el cargador.
- `malloc()` retorna una dirección *por arriba* del segmento de datos: es el **heap**. `sbrk(0)` devuelve el límite superior actual del heap (*program break*).
- `&local` y `&argc` están en la dirección más alta: el **stack**, que crece hacia abajo. En Linux x86-64, suele empezar cerca de `0x7fff\dots`.

4.4. Verificación con `/proc/<pid>/maps`

El layout puede contrastarse con la información que el kernel exporta:

```
$ cat /proc/$(pidof a.out)/maps
55a3f4d2b000-55a3f4d2c000 r-xp 00000000 fd:01 ... /home/u/a.out <- .text
55a3f4d2c000-55a3f4d2d000 r--p 00001000 ... <- .rodata
55a3f4d2d000-55a3f4d2e000 r--p 00002000 ... <- .data ro
55a3f4d2e000-55a3f4d2f000 rw-p 00003000 ... <- .data rw + .bss
55a3f5e6b000-55a3f5e8d000 rw-p 00000000 00:00 0 [heap]
7f... -7f... r-xp ... libc.so.6
7ffe89a1c000-7ffe89a3d000 rw-p 00000000 00:00 0 [stack]
```

Idea intuitiva

La maquinaria de paginado le permite al sistema dejar *huecos* gigantes entre **heap** y **stack**, sin reservar memoria física por ellos: los *pte* intermedios son inválidos ($V = 0$) y sólo se materializan en cuanto el proceso los referencie. Esto es lo que viabiliza el crecimiento dinámico del stack y del heap.

5. Ejercicio 4 — Arquitectura de 16 bits con páginas de 1 KB

Enunciado. Arquitectura con espacio de direcciones de 16 bits, paginado con páginas de 1 KB.

5.1. 4.a — Capacidad máxima de la memoria

Con direcciones de 16 bits, el espacio direccionable tiene $2^{16} = 65\,536$ bytes, es decir:

$$\text{capacidad máxima} = 2^{16} \text{ bytes} = 64 \text{ KB.}$$

5.2. 4.b — Cantidad de páginas

Con páginas de 1 KB = 2^{10} bytes:

$$\text{número de páginas} = \frac{2^{16}}{2^{10}} = 2^6 = \boxed{64 \text{ páginas.}}$$

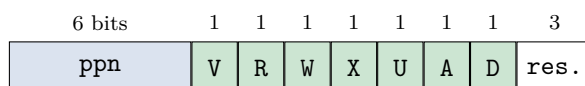
5.3. 4.c — Representación de la tabla de páginas y de la *pte*

Una dirección lógica de 16 bits se divide en:



- **vpn** (6 bits): índice en la tabla de páginas ($2^6 = 64$ entradas).
- **offset** (10 bits): desplazamiento dentro de la página de 1 KB.

Formato propuesto de *pte*. Cada entrada describe dónde está la página física y con qué permisos. Un diseño compacto de 16 bits sería:



Los campos son los habituales (cf. resumen teórico de gestión de memoria, formato *pte* de RV-32):

- **ppn** (6 bits): número de página física (*physical page number*).
- **V**: bit *valid*. Si $V = 0$, indexar esa entrada provoca un *page fault*.
- **R, W, X**: permisos de *read*, *write* y *execute*.
- **U**: permiso en modo *user*.
- **A**: encendido por el hardware cuando se accede (lectura o escritura).
- **D**: encendido por el hardware cuando se escribe (*dirty*).

Tamaño total de la tabla. $64 \text{ entradas} \times 16 \text{ bits} = 1024 \text{ bits} = 128 \text{ bytes}$, lo que cabe holgadamente en una sola página de 1 KB.

5.4. 4.d — Tabla de páginas de un proceso concreto

El proceso tiene:

- 1800 bytes de **código**;
- 600 bytes de **datos** (a continuación);
- una **página de guarda** entre datos y pila (para detectar *stack overflow*);
- 1 KB de **pila**.

Cuántas páginas usa cada área.

código : $\lceil 1800/1024 \rceil = 2$ páginas	(páginas 0 y 1)
datos : $\lceil 600/1024 \rceil = 1$ página	(página 2)
guarda : 1 página inválida	(página 3)
pila : $\lceil 1024/1024 \rceil = 1$ página	(página 4)

El espacio de direcciones del proceso queda $[0, 5 \text{ KB}]$. Las entradas 5 a 63 no se usan ($V=0$).

Tabla de páginas. Los **ppn** concretos los elige el SO según los *frames* libres; en lo que sigue se usan 0x07, 0x12, 0x1A y 0x2F a modo de ejemplo.

#	Área	ppn	V	R	W	X	U	D/A	Comentario
0	código	0x07	1	1	0	1	1	–	primera página de <code>.text</code>
1	código	0x12	1	1	0	1	1	–	segunda página de <code>.text</code>
2	datos	0x1A	1	1	1	0	1	–	<code>.data</code> + <code>.bss</code>
3	guarda	—	0	0	0	0	0	–	cualquier acceso $\Rightarrow$ <i>page fault</i>
4	pila	0x2F	1	1	1	0	1	–	1 KB de stack
5–63	—	—	0	0	0	0	0	–	páginas no asignadas

página 0 -- código	0x0000
página 1 -- código	0x0400
página 2 -- datos	0x0800
página 3 -- guarda (V=0)	0x0C00
página 4 -- pila	0x1000 0x1400

Idea intuitiva

Si el stack crece más allá de la página 4, la siguiente referencia cae en la página 3 (guarda, $V = 0$) y el hardware genera un *page fault*. El kernel reconoce el patrón “escritura a una página de guarda inválida” y aborta el proceso con *stack overflow*. Es exactamente el mecanismo descrito en el resumen teórico de gestión de memoria.

6. Ejercicio 5 — Tablas de páginas para 4 GB en 32 bits

Enunciado. En un SO sobre una arquitectura de 32 bits con paginado, determinar la cantidad de *ptes* y la memoria total necesaria para cubrir 4 GB de espacio de direcciones, con (a) páginas de 4 KB y (b) páginas de 4 MB.

Se asume el tamaño habitual de cada *pte*: 32 bits = 4 bytes (espacio para el **ppn** más los flags). El cálculo general es

$$\#ptes = \frac{2^{32}}{\text{tamaño página}}, \quad \text{memoria de la tabla} = \#ptes \times 4 \text{ bytes}.$$

6.1. 5.a — Páginas de 4 KB

$$\#ptes = \frac{2^{32}}{2^{12}} = 2^{20} = 1\,048\,576$$

$$\text{memoria de la tabla} = 2^{20} \times 4 \text{ B} = 2^{22} \text{ B} = 4 \text{ MB}.$$

Una tabla *plana* de 4 MB **por proceso** es enorme y exigiría un bloque contiguo de RAM grande para alojarla. Por eso los SO modernos representan la tabla como un **árbol de varios niveles** (por ejemplo, 2 niveles en RV-32, 4 niveles en x86-64): sólo se reservan los nodos correspondientes a regiones del espacio de direcciones efectivamente usadas, y los enormes huecos no consumen memoria.

6.2. 5.b — Páginas de 4 MB

$$\#ptes = \frac{2^{32}}{2^{22}} = 2^{10} = 1024$$

$$\text{memoria de la tabla} = 2^{10} \times 4 \text{ B} = 2^{12} \text{ B} = 4 \text{ KB}.$$

Una tabla de sólo 4 KB cabe en una sola página, lo que simplifica muchísimo la administración. El costo es la **fragmentación interna**: cualquier asignación de memoria, por pequeña que sea, ocupa 4 MB completos.

Resumen comparativo

Caso	Tamaño de página	#ptes	Memoria de la tabla	Comentario
(a)	4 KB	$2^{20} = 1.048.576$	4 MB	menor frag. interna; tabla enorme
(b)	4 MB	$2^{10} = 1.024$	4 KB	tabla chica; gran frag. interna

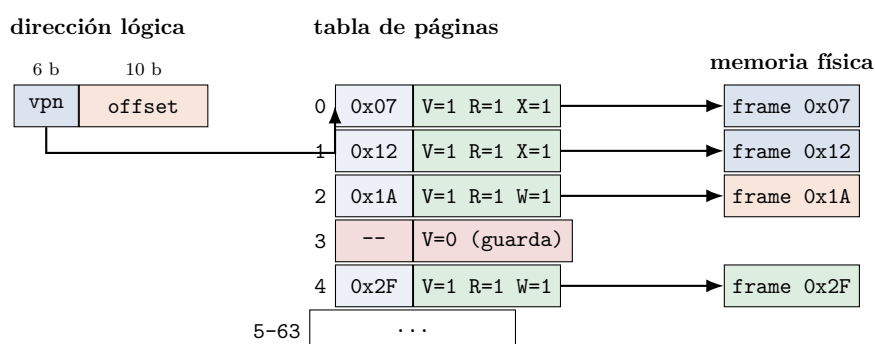
Importante

Hay un compromiso fundamental: **páginas más chicas** dan menor fragmentación interna pero tablas mucho más grandes; **páginas más grandes** (*huge pages*) reducen el tamaño de la tabla y la presión sobre el TLB, pero malgastan memoria si los pedidos son pequeños. Los SO modernos combinan ambos: páginas de 4 KB por default, con la opción de usar páginas de 2 MB / 1 GB (*huge pages*) en regiones específicas.

7. Ejercicio 6 — Representación de la tabla de páginas de un proceso

Enunciado. Representar la tabla de páginas de un proceso.

Se toma el proceso del Ejercicio 4d (el ejercicio del práctico no fija un proceso particular, así que se reutiliza ese ejemplo para concretar). La representación enfatiza la traducción de una dirección lógica a una dirección física a través de la *pte* indexada.



Traducción de una dirección lógica. Dada la dirección lógica *va*, se separa en (*vpn*, *offset*) con los 6 bits altos y 10 bits bajos. Se accede a la entrada *vpn* de la tabla. Si *V* = 1 y los permisos cuadran con el tipo de acceso, la dirección física queda

$$pa = \text{pagetable}[vpn].ppn \times 1024 + \text{offset}.$$

Si *V* = 0 o se viola algún permiso, el hardware lanza un *page fault*.

Ejemplo numérico. La instrucción referencia la dirección lógica 0x0814 (en el área de datos). Se descompone como $vpn = 2$ y $offset = 0x014 = 20$. La pte 2 tiene $ppn = 0x1A$ y $V = 1$, $R = 1$, $W = 1$. La dirección física es

$$0x1A \times 1024 + 20 = 26\,624 + 20 = 26\,644 = 0x6814.$$

8. Ejercicio 7 — Memoria compartida entre dos procesos

Enunciado. Dos procesos comparten un área de memoria de 2 páginas. En el primer proceso el área se mapea a partir de 0xA000; en el segundo desde 0xF000. Mostrar con un diagrama el layout de memoria de cada proceso y las tablas de páginas. Las páginas son de 4 KB.

8.1. Análisis de las direcciones lógicas

Con páginas de $4\text{ KB} = 2^{12}$, el $offset$ ocupa los 12 bits menos significativos y el vpn ocupa los bits altos. Por lo tanto:

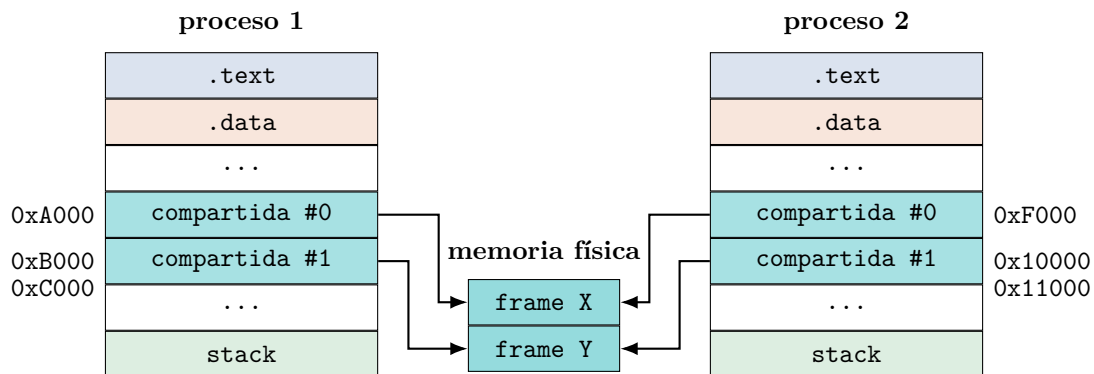
Dirección	vpn	offset
0xA000	0xA = 10	0
0xB000	0xB = 11	0
0xF000	0xF = 15	0
0x10000	0x10 = 16	0

Proceso 1. El área compartida ocupa las páginas lógicas 10 y 11 (rango [0xA000, 0xC000)).

Proceso 2. El área compartida ocupa las páginas lógicas 15 y 16 (rango [0xF000, 0x11000)).

En ambos procesos, esas $ptes$ apuntan a los **mismos dos frames físicos X e Y**.

8.2. Diagrama



8.3. Tablas de páginas

Proceso 1						Proceso 2
vpn	decimal	ppn	V	R	W	
...	...	...	...	...	...	
0xA	10	X	1	1	1	
0xB	11	Y	1	1	1	
...	...	...	...	...	...	
vpn	decimal	ppn	V	R	W	
...	...	...	...	...	...	
0xF	15	X	1	1	1	
0x10	16	Y	1	1	1	
...	...	...	...	...	...	

Idea intuitiva

La clave del esquema es que *distintas* entradas ($\text{vpn} = 10$ en P1, $\text{vpn} = 15$ en P2) almacenan *el mismo* ppn . Las direcciones lógicas son distintas en cada proceso, pero el hardware traduce ambas al mismo *frame* físico. Así, una escritura del proceso 1 a $0xA000$ es inmediatamente visible para el proceso 2 leyendo $0xF000$. Es exactamente el mecanismo que usan `shmat` y `mmap` con `MAP_SHARED`.

Importante

La memoria compartida no es un canal sincronizado por sí mismo: si ambos procesos escriben simultáneamente al área compartida, pueden producirse *race conditions*. Es responsabilidad del programador agregar un mecanismo de sincronización (semáforos, mutex en memoria compartida, `futex`, etc.).

9. Ejercicio 8 — mmap de un archivo

Enunciado. Compilar y ejecutar (en Linux) el archivo `mmapfile.c` dado. (a) ¿Por qué `data + 7000` no causa *segmentation fault*, si el archivo ocupa poco más de 4096 bytes? (b) Modificar el programa para mapear en lectura y escritura, con persistencia automática al `munmap`; ¿cómo lo hace el SO? (c) Verificar el tamaño del archivo después de ejecutar.

9.1. Listado del programa original

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>

#define PGSIZE 4096
#define filename "mmapfile.txt"

int createfile(void) {
    int f = open(filename, O_RDONLY);
    if (f < 0) {
        const char *str1 = "Hello world in page 1";
        const char *str2 = "I'm in second page";
        const unsigned char zero = 0;
        int l = strlen(str1);
        f = open(filename, O_RDWR | O_CREAT, 0666);
        write(f, str1, l);
        for (i = 1; i < PGSIZE; i++) write(f, &zero, 1);
        write(f, str2, strlen(str2) + 1);
        close(f);
        f = open(filename, O_RDONLY);
    }
    return f;
}

int main(void) {
    int f = createfile();
    char *data = mmap(0, PGSIZE * 2, PROT_READ, MAP_PRIVATE, f, 0);
    if (data) {
        printf("file contents mapped at %p\n", data);
        printf("%s\n", data);
        printf("%s\n", data + 4096);
        printf("%s\n", data + 7000); /* acceso valido aunque el archivo "termine" antes */
    }
}
```

```

    munmap(data, PGSIZE * 2);
} else {
    printf("mmap error\n");
}
close(f);
return 0;
}

```

9.2. Compilación y ejecución

```

$ gcc -Wall -O0 -o mmapfile mmapfile.c
$ ./mmapfile
file contents mapped at 0x7f8e2c1c4000
Hello world in page 1
I'm in second page

```

<- aparece linea en blanco (los bytes son ceros)

9.3. 8.a — ¿Por qué `data + 7000` no causa `segfault`?

El archivo `mmapfile.txt` pesa `strlen(str1) + (PGSIZE - strlen(str1)) + strlen(str2) + 1` ≈ 4115 bytes, es decir, poco más de una página.

Sin embargo, la llamada `mmap(0, PGSIZE * 2, ..., f, 0)` pidió mapear **dos** páginas (8192 bytes) a partir del offset 0.

Importante

La región mapeada es *redondeada hacia arriba al tamaño de página*. Por eso `mmap` reserva 2 páginas válidas en el espacio de direcciones, aunque el archivo no llene la segunda completamente.

El kernel, en ese caso, **rellena con ceros** la parte de la última página que excede al EOF del archivo, según lo definido por POSIX. Por lo tanto, todas las direcciones del rango `[data, data + 8192)` son válidas y, en particular, `data + 7000`:

- Está *dentro* de la región mapeada (offsets `[0, 8192)`): la **pte** correspondiente está definida.
- Los bytes leídos a partir de allí son los del archivo en el offset `7000 - 4096 = 2904` de la página 1. Como el archivo no llega hasta ahí, son **ceros** (rellenados por el kernel).

Por eso `printf("%s\n", data + 7000)` imprime una cadena vacía (el primer byte es `\0`) en lugar de provocar un *segmentation fault*.

Idea intuitiva

Si en cambio se accediese a `data + 8192` (la página 2, no mapeada), la **pte** no existiría y sí se produciría *segfault* (o más precisamente, *SIGSEGV*). El mecanismo exacto es el visto en el resumen teórico de memoria virtual: el hardware levanta un *page fault*, el kernel detecta que la dirección no pertenece a ningún VMA del proceso y, al no ser recuperable, envía la señal correspondiente.

9.4. 8.b — Modificación para lectura y escritura con persistencia

Hay que cambiar dos cosas en la llamada a `mmap`:

1. Permisos de la página: `PROT_READ` $\rightarrow$ `PROT_READ | PROT_WRITE`.
2. Modo de compartición: `MAP_PRIVATE` $\rightarrow$ `MAP_SHARED`.

Importante

Con `MAP_PRIVATE` las escrituras al área mapeada generan una copia privada (*copy on write*) y *no* se propagan al archivo. Para que las modificaciones queden en el archivo se necesita `MAP_SHARED`. Además, el archivo debe haberse abierto en modo `O_RDWR`.

```
int main(void) {
    int f = createfile_rw(); // abre con O_RDWR | O_CREAT
    char *data = mmap(0, PGSIZE * 2, PROT_READ | PROT_WRITE,
                      MAP_SHARED, f, 0);
    if (data == MAP_FAILED) { perror("mmap"); return 1; }

    printf("file contents mapped at %p\n", data);
    printf("Before: %s\n", data);

    /* modifico el contenido a traves de la memoria mapeada */
    const char *nuevo = "Hola desde mmap()!";
    memcpy(data, nuevo, strlen(nuevo) + 1);

    /* munmap se encarga de hacer flush de las paginas dirty al archivo */
    munmap(data, PGSIZE * 2);
    close(f);
    return 0;
}
```

Cómo lo hace el SO. La explicación está en el resumen teórico de memoria virtual:

1. Por cada página mapeada, el hardware mantiene el bit **D** (*dirty*) en la **pte**. Cada vez que el proceso escribe en esa página, el D pasa a 1 automáticamente.
2. Cuando se invoca `munmap` (o cuando el sistema hace un *flush* periódico, p.ej. `pdflush`), el kernel recorre las **pte** del mapping y escribe al archivo *sólo las páginas con D=1*, en el offset correspondiente. Las páginas con D=0 no se tocan, ahorrando E/S.
3. Finalmente, la región del espacio de direcciones se libera y las **pte** pasan a $V = 0$.

Adicionalmente, la llamada `msync(addr, len, MS_SYNC)` permite forzar la sincronización a disco *sin* desmontar el mapping; es útil cuando se quiere garantizar persistencia antes del `munmap` (por ejemplo, frente a un eventual crash).

9.5. 8.c — Tamaño del archivo después

```
$ ls -l mmapfile.txt
-rw-r--r-- 1 tomi tomi 4115 may 20 23:01 mmapfile.txt
$ stat -c "size: %s bytes" mmapfile.txt
size: 4115 bytes
```

El tamaño no cambia. Aunque el mapping cubre dos páginas (8192 bytes), el archivo subyacente sigue ocupando los ~ 4115 bytes que tenía antes. Las escrituras realizadas en offsets ≥ 4115 dentro de la primera página se propagan a esa zona del archivo siempre que no excedan el tamaño actual; las escrituras dentro de la segunda página *sí* podrían propagarse, pero el comportamiento ante escrituras más allá del EOF en un mapping de un archivo regular no está definido por POSIX (en Linux suele generar `SIGBUS`).

Importante

Para extender el archivo, hay que llamar a `ftruncate(f, new_size)` *antes* de `mmap`. Sólo entonces la región mapeada coincide con un archivo de ese tamaño y las escrituras quedan persistidas correctamente.

10. Ejercicio 9 — Memoria compartida IPC con `shmwriter` y `shmreader`

Enunciado. Compilar y ejecutar (en ese orden) `shmwriter.c` y `shmreader.c` para observar cómo funciona la API de *shared memory* (IPC de UNIX). (a) Volver a correr `shmreader` y ver qué ocurre. (b) Comentar la línea `shmctl(..., IPC_RMID, NULL)` de `shmreader.c`; correr `shmwriter` y luego `shmreader` varias veces. (c) Modificar ambos programas para leer/escribir en la dirección lógica `str+500`.

10.1. Cómo funciona la API

- `key_t ftok(path, id)`: deriva una clave numérica a partir de un archivo existente y un `id` (un byte). Los procesos cooperantes acuerdan el mismo `path` y `id` para coincidir en la misma clave.
- `int shmget(key, size, flags)`: obtiene (o crea con `IPC_CREAT`) un segmento de *shared memory* con esa clave y tamaño. Devuelve un identificador `shmid`.
- `void *shmat(shmid, addr, flags)`: mapea el segmento en el espacio de direcciones del proceso. Si `addr=0`, el kernel elige la dirección.
- `int shmdt(addr)`: hace *detach*: quita el mapping de este proceso, pero el segmento sigue existiendo.
- `int shmctl(shmid, IPC_RMID, NULL)`: marca el segmento para destrucción. Se libera cuando el último proceso hace `shmdt`.

10.2. Programas provistos

Listing 1: `shmwriter.c`

```
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <string.h>

int main(void) {
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    char *str = (char *)shmat(shmid, (void *)0, 0);

    printf("shared memory logical address: %p\n", str);
    printf("Data to write to shmem: ");
    fgets(str, 1024, stdin);
    str[strcspn(str, "\n")] = '\0';
    printf("Data written in memory: %s\n", str);

    shmdt(str);
    return 0;
}
```

Listing 2: shmreader.c

```
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>

int main(void) {
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    char *str = (char *)shmat(shmid, (void *)0, 0);

    printf("shared memory logical address: %p\n", str);
    printf("Data read from memory: %s\n", str);

    shmdt(str);
    shmctl(shmid, IPC_RMID, NULL); /* destruye el segmento */
    return 0;
}
```

10.3. Ejecución base

```
$ touch shmfile
$ gcc -o shmwriter shmwriter.c
$ gcc -o shmreader shmreader.c

$ ./shmwriter
shared memory logical address: 0x7fbcde47b000
Data to write to shmem: Hola desde shmwriter
Data written in memory: Hola desde shmwriter

$ ./shmreader
shared memory logical address: 0x7f0123abc000
Data read from memory: Hola desde shmwriter
```

Observación. Las direcciones lógicas reportadas son *diferentes* (0x7fbc... vs 0x7f01...), pero los datos coinciden: ambos procesos están viendo los mismos *frames* físicos a través de *vpn* distintos. Es exactamente el escenario del Ejercicio 7.

10.4. 9.a — Volver a correr shmreader

Si *shmreader* *destruye* el segmento al terminar (con `shmctl(IPC_RMID)`, como está escrito arriba), la segunda ejecución encuentra el siguiente escenario:

- El segmento anterior ya no existe (fue marcado para eliminación y todos los procesos hicieron `shmdt`).
- `shmget` con `IPC_CREAT` crea uno **nuevo** con la misma clave, pero su contenido es *indeterminado* (típicamente, todo ceros en Linux).
- `printf("Data read from memory: %s\n", str)` imprime una línea vacía.

```
$ ./shmreader # primera vez: ve los datos
shared memory logical address: 0x7f...
Data read from memory: Hola desde shmwriter

$ ./shmreader # segunda vez: segmento recién creado, vacío
shared memory logical address: 0x7f...
Data read from memory:
```

10.5. 9.b — Comentar la destrucción y correr varias veces

```
// shmctl(shmid, IPC_RMID, NULL); <-- comentada
```

Con esta modificación, el segmento *persiste* entre ejecuciones. Se mantiene activo en el kernel hasta que se borre explícitamente con `ipcrm` o se reinicie el sistema:

```
$ ./shmwriter
Data to write to shmem: mensaje persistente
Data written in memory: mensaje persistente

$ ./shmreader
Data read from memory: mensaje persistente

$ ./shmreader # de nuevo: los datos siguen ahí
Data read from memory: mensaje persistente

$ ./shmreader
Data read from memory: mensaje persistente

$ ipcs -m # ver segmentos de memoria compartida activos
----- Memoria compartida -----
key shmid propietario perms bytes nattch
0x4180... 98305 tomi 666 1024 0
```

Para limpiar manualmente:

```
$ ipcrm -m 98305
```

Idea intuitiva

Esto ilustra una propiedad clave de la API: los segmentos IPC de System V tienen **ciclo de vida del kernel**, no del proceso. Sobreviven al cierre de todos los procesos que los usan, salvo que se los elimine explícitamente. Es una diferencia importante con respecto a `mmap MAP_ANONYMOUS` o `shm_open + mmap` (POSIX), cuyo ciclo de vida se administra de manera más parecida a la de un archivo.

10.6. 9.c — Lectura/escritura en `str + 500`

Se trata de usar como base del dato compartido el offset 500 dentro del segmento. La modificación es simple y se aplica en ambos lados.

shmwriter (modificado).

```
char *str = (char *)shmat(shmid, (void *)0, 0);
printf("shared memory logical address: %p\n", str);
printf("Escribiendo en offset 500 (str+500 = %p):\n", str + 500);
printf("Data to write: ");
fgets(str + 500, 1024 - 500, stdin);
(str + 500)[strcspn(str + 500, "\n")] = '\0';
printf("Data written in memory at str+500: %s\n", str + 500);
shmdt(str);
```

shmreader (modificado).

```
char *str = (char *)shmat(shmid, (void *)0, 0);
```

```
printf("shared memory logical address: %p\n", str);
printf("Leyendo desde offset 500 (str+500 = %p):\n", str + 500);
printf("Data read from memory: %s\n", str + 500);
shmdt(str);
```

Notas.

- `str + 500` es una dirección lógica válida porque el segmento tiene 1024 bytes y $500 < 1024$.
- La aritmética de punteros funciona porque `str` es `char *`: `str + 500` avanza exactamente 500 bytes.
- En el caso del `writer` se acota `fgets(str + 500, 1024 - 500, stdin)` para que no se escriba más allá del fin del segmento.
- Los primeros 500 bytes del segmento siguen sin tocar; si algún programa anterior los había escrito, su contenido permanece.

Resumen del ejercicio

- La API `shmget/shmat/shmdt/shmctl` expone explícitamente la abstracción de *región de memoria compartida* entre procesos.
- Cada proceso mapea la misma región a (posiblemente) *distintas direcciones lógicas*, pero el contenido es compartido porque las `pte` apuntan a los mismos frames físicos.
- El ciclo de vida del segmento depende del sistema, no de los procesos: si nadie llama a `shmctl(IPC_RMID)`, el segmento queda hasta el reboot.
- Es responsabilidad del programador sincronizar accesos al área compartida (semáforos, mutex en *shm*, etc.).

11. Conclusiones

A lo largo del práctico se recorrió, ejercicio a ejercicio, toda la maquinaria que el sistema operativo usa para gestionar la memoria:

- Los **Ejercicios 1 y 3** mostraron el *lado práctico*: cómo medir el consumo real de memoria desde la línea de comandos y desde un programa propio, distinguiendo `VIRT/RES/PSS/USS` y los distintos segmentos del proceso (`.text`, `.data`, heap, stack).
- Los **Ejercicios 2, 4, 5 y 6** se concentraron en la *asignación* y el *paginado*: el *buddy system* ilustra los compromisos entre fragmentación interna y externa; el cálculo de la tabla de páginas en arquitecturas de 16 y 32 bits muestra por qué hacen falta tablas multinivel y/o *huge pages* en sistemas reales.
- Los **Ejercicios 7, 8 y 9** aplicaron la maquinaria al problema de *compartir memoria entre procesos* y mapear archivos en memoria. Los detalles relevantes (bit `D` para persistencia de `mmap`; ciclo de vida del segmento de *shared memory*; visión común de distintos `vpn` sobre los mismos `ppn`) son consecuencia directa del modelo del capítulo teórico.

Idea intuitiva

Lo que une todos los ejercicios es la *misma* abstracción: la tabla de páginas. Sirve para confinar un proceso, compartir memoria, mapear archivos, hacer crecer el stack y abaratar `fork()`. La diferencia entre cada caso está sólo en cómo el SO configura las `pte` y reacciona ante *page faults*, no en una maquinaria nueva por feature.